

Full SRNN Model Specification

This document is a complete, code-faithful specification of the SRNN model implemented in `train_srnn/models/srnn_cell.py` (single `SRNNCell` and K-batched `BatchedSRNNCell`) wrapped by `train_srnn/models/sequence_model.py`. All formulas use the exact transforms applied in code; symbols in math match the parameter names in the saved checkpoints.

Convention: K = number of ablation variants packed into a `BatchedSRNNCell` (in single-cell mode, drop the leading K dimension), B = batch, T = sequence length, $N = \text{num_units}$, $n_E = \lfloor N/2 \rfloor$, $n_I = N - n_E$, $D = \text{input_size}$, O = output dim, E = number of output (readout) neurons after I/O masking.

1. State variables

The cell carries five state components, packed flat per batch element as $[a_E | a_I | b_E | b_I | x]$:

symbol	meaning	shape (single)	shape (batched)	init
x	dendritic potential	(B, N)	(K, B, N)	$0.1 \cdot \mathcal{N}(0, 1)$ if not <code>TrainableIC</code>
a_E	SFA (E side), n_{aE} tiers	(B, n_E, n_{aE})	$(K, B, n_E, n_{aE}^{\max})$	0
a_I	SFA (I side), n_{aI} tiers	(B, n_I, n_{aI})	$(K, B, n_I, n_{aI}^{\max})$	0
b_E	STD (E side), available fraction	(B, n_E)	(K, B, n_E)	1
b_I	STD (I side)	(B, n_I)	(K, B, n_I)	1

In `BatchedSRNNCell`, `max_n_a_E` / `max_n_b_E` etc. are the maxima over all packed variants; per-variant active tiers are masked at compute time (see `sfa_E_mask`, `std_E_mask` below).

2. Learnable parameters (raw)

All listed parameters are saved in `model_state_dict` under the names below (with `cell.` prefix). Shapes use the batched layout; for single `SRNNCell`, remove the leading K dimension and (for SFA vec params) the trailing broadcasted axis.

2.1 Recurrent + input weights

parameter	shape	init
<code>W_raw</code>	(K, N, N)	from <code>RMTMatrix</code> (random matrix theory init, real-valued)
<code>W_in</code>	(K, N, D)	$\mathcal{N}(0, 0.1^2)$
<code>W_raw_gain</code>	$(K,)$	1
<code>W_in_gain</code>	$(K,)$	1

2.2 Threshold

parameter	shape	init
<code>a_0_vec</code>	(K, N)	0.35 (constant)
<code>a_0_scalar</code>	$(K,)$	0

The single-cell version uses `a_0` of shape $(N,)$ with init 0.35 and no scalar.

2.3 Global timescale

parameter	shape	init
log_tau_global	$(K,)$	$\text{softplus}^{-1}(\tau_g^{\text{init}})$, default $\tau_g^{\text{init}} = 1$

2.4 Dendritic time constant

parameter	shape	init
log_tau_d_vec	(K, N)	$\text{softplus}^{-1}(0.1)$
log_tau_d_gain	$(K,)$	0

2.5 SFA parameters (E side, present iff $n_{aE} > 0$)

parameter	shape	init
log_tau_a_E_vec	$(K, n_E, n_{aE}^{\text{max}})$	per-tier interpolation in inv-softplus space between $\text{softplus}^{-1}(0.25)$ and $\text{softplus}^{-1}(10)$, equally spaced over $j = 0, \dots, n_{aE} - 1$; 0 for inactive tiers
log_tau_a_E_gain	$(K,)$	0
log_c_E_vec	$(K, n_E, n_{aE}^{\text{max}})$	$\text{softplus}^{-1}(0.05)$
log_c_E_gain	$(K,)$	0
c_0_E_vec	$(K, n_E, n_{aE}^{\text{max}})$	0
c_0_E_scalar	$(K,)$	0

In the *single SRNNCell* for $n_{aE} = 1$ the param is `log_tau_a_E` of shape $(\text{base}_E, 1)$ where $\text{base}_E = (n_E,)$ if `per_neuron` else $(1,)$. For $n_{aE} \geq 2$, the single cell stores only endpoints `log_tau_a_E_lo`, `log_tau_a_E_hi` and interpolates in log space at runtime — see §4.4.

2.6 SFA parameters (I side, present iff $n_{aI} > 0$)

Same structure as §2.5, with $n_E \rightarrow n_I$ and `_E` $\rightarrow$ `_I`.

2.7 STD parameters (E side, present iff $n_{bE} > 0$)

parameter	shape	init
log_tau_b_rec_E_vec	(K, n_E)	$\text{softplus}^{-1}(1.0)$
log_tau_b_rec_E_gain	$(K,)$	0
log_tau_b_rel_E_vec	(K, n_E)	$\text{softplus}^{-1}(0.25)$
log_tau_b_rel_E_gain	$(K,)$	0

2.8 STD parameters (I side, present iff $n_{bI} > 0$)

Same structure as §2.7, with $n_E \rightarrow n_I$.

2.9 Readout (in `SequenceModel`)

parameter	shape	init
readout_weight	(K, O, E)	Kaiming-uniform per slice k (PyTorch <code>nn.Linear</code> default with $a = \sqrt{5}$)
readout_bias	$(K, 1, O)$	0

Single-cell mode uses a single `nn.Linear(E, 0)`.

2.10 Trainable initial condition (in SequenceModel, optional)

parameter	shape	init
ic.ic	(K, S) where $S = \text{state_size}$	0 initially; set by burn-in if enabled

3. Buffers (non-trainable, shape-as-stored)

buffer	shape	semantics
sparsity_masks	(K, N, N)	$\{0, 1\}$ — fixed connectivity mask from RMT init
dales_signs	(K, N)	± 1 — column-wise: +1 for E source, -1 for I source
dales_mask	$(K, 1, 1)$	$\{0, 1\}$ — whether Dale’s law is enforced for variant k
echo_flags	$(K, 1, 1)$	$\{0, 1\}$ — whether W_{raw} is frozen (reservoir)
_echo_grad_mask	$(K, 1, 1)$	1 — echo_flags , multiplied into $\nabla_{W_{\text{raw}}}$ via register_hook
skip_flags	$(K,)$	$\{0, 1\}$ — output residual; see §7
W_in_mask	$(1, N, 1)$	$\{0, 1\}$ — input-neuron partition (only $\sim 25\%$ rows are 1)
output_mask	$(N,)$	$\{0, 1\}$ — readout-neuron partition (only $\sim 25\%$ entries are 1, E in shape table)
input_mask	$(N,)$	$\{0, 1\}$ — same partition info; informational
sfa_E_mask	$(K, 1, n_{aE}^{\max})$	$\{0, 1\}$ per tier — 1 for active tiers of variant k , 0 otherwise
sfa_I_mask	$(K, 1, n_{aI}^{\max})$	analogous
std_E_mask	$(K, 1)$	$\{0, 1\}$ — whether STD-E is active for variant k
std_I_mask	$(K, 1)$	analogous
_a_0_vec_mask, _log_tau*_vec_mask, _log_c*_vec_mask, _c_0*_vec_mask	per-param, broadcast-ready	per-variant per_neuron flag (1 for per-neuron, 0 otherwise), multiplied into the corresponding gradient via register_hook so that non-per-neuron variants keep their _vec params frozen at init
readout_ids	$(K,)$	$\{0, 1, 2\}$ — synaptic / rate / dendritic readout selector per variant

4. Effective parameters (post-transform — what enters the RHS)

These are the values reported in the per-variant `param_table_*.txt`.

4.1 Recurrent weight

For variant k , with $\mathbf{1}_{\text{Dale}} \equiv \text{dales_mask}[k]$, $\mathbf{s} \equiv \text{dales_signs}[k] \in \{\pm 1\}^N$, $\mathbf{M} \equiv \text{sparsity_masks}[k] \in \{0, 1\}^{N \times N}$, and $g_W \equiv W_{\text{raw_gain}}[k]$:

$$W_{ij}^{\text{eff}} = g_W \cdot \mathbf{M}_{ij} \cdot \left[\mathbf{1}_{\text{Dale}} s_j \text{softplus}(W_{ij}^{\text{raw}}) + (1 - \mathbf{1}_{\text{Dale}}) W_{ij}^{\text{raw}} \right].$$

Index j is the **source** neuron. Dale’s law is enforced column-wise: every outgoing weight from neuron j shares the sign s_j . With Dale’s on, $W_{:,j}^{\text{eff}} \geq 0$ for $s_j = +1$ (E source) and $W_{:,j}^{\text{eff}} \leq 0$ for $s_j = -1$ (I source); off-mask entries are exactly 0.

4.2 Input weight

$$W^{\text{in,eff}} = g_{W_{\text{in}}} \cdot W^{\text{in}} \cdot M^{\text{in}}, \quad M_{i,:}^{\text{in}} = W_{\text{in_mask}}[i].$$

`W_in_mask` zeros out rows for non-input neurons (effectively $\sim 75\%$ of rows are zero), so $W^{\text{in,eff}}$ is non-trivial only on the input neuron partition.

4.3 Time constants

Define the per-variant global rescale $\tau_g(k) \equiv \text{softplus}(\text{log_tau_global}[k])$.

Dendritic:

$$\tau_d(k, i) = \tau_g(k) \cdot \exp(\text{log_tau_d_gain}[k]) \cdot \text{softplus}(\text{log_tau_d_vec}[k, i]).$$

SFA E (per tier j , batched form):

$$\tau_{a,E}(k, i, j) = \tau_g(k) \cdot \exp(\text{log_tau_a_E_gain}[k]) \cdot \text{softplus}(\text{log_tau_a_E_vec}[k, i, j]).$$

The vec is initialized so that, for a variant with $n_{aE} \geq 2$, the j -th tier sits at a value $\bar{\tau}_j$ with $\text{softplus}(\bar{\tau}_j)$ linearly interpolating in inv-softplus space between 0.25 and 10 s; tiers above $n_{aE} - 1$ are inactive (`sfa_E_mask[k, 0, j] = 0`).

SFA E (single-cell form, multi-timescale): When $n_{aE} \geq 2$ the *single* SRNNCell instead stores only endpoints $\tau_{a,E}^{\text{lo}} = \text{softplus}(\text{log_tau_a_E_lo})$, $\tau_{a,E}^{\text{hi}} = \text{softplus}(\text{log_tau_a_E_hi})$ and interpolates in **log space** at runtime,

$$\tau_{a,E}^{(j)} = \tau_g \cdot \exp\left(\log \tau_{a,E}^{\text{lo}} + \frac{j}{n_{aE}-1} (\log \tau_{a,E}^{\text{hi}} - \log \tau_{a,E}^{\text{lo}})\right), \quad j = 0, \dots, n_{aE} - 1.$$

SFA I, STD E, STD I: identical structure to SFA E vec/gain.

$$\tau_{a,I}, \quad \tau_{b,\text{rec},E}, \quad \tau_{b,\text{rel},E}, \quad \tau_{b,\text{rec},I}, \quad \tau_{b,\text{rel},I}$$

all follow $\tau = \tau_g \cdot \exp(\text{gain}) \cdot \text{softplus}(\text{vec})$.

4.4 SFA coupling and offset

$$c_E(k, i, j) = \exp(\text{log_c_E_gain}[k]) \cdot \text{softplus}(\text{log_c_E_vec}[k, i, j]), \quad c_I \text{ analogously.}$$

$$c_{0,E}(k, i, j) = \text{c_0_E_vec}[k, i, j] + \text{c_0_E_scalar}[k], \quad c_{0,I} \text{ analogously.}$$

4.5 Threshold

$$a_0(k, i) = \text{a_0_vec}[k, i] + \text{a_0_scalar}[k].$$

5. Right-hand side (continuous-time ODE)

Run the cell at simulated time-step $\Delta t = h/n_{\text{ode}}$ (with `h = config.h` and $n_{\text{ode}} = \text{ode_unfolds}$ sub-steps per cell call).

5.1 External drive

$$u(t) = W^{\text{in,eff}} x_{\text{input}}(t) \in \mathbb{R}^N,$$

i.e. $(B, N) = (B, D) @ (W_{\text{in_eff}})^T$, batched over K in the batched cell.

5.2 Effective potential after SFA subtraction

For neurons in the E partition (index range $0 : n_E$), with $a_E \in \mathbb{R}^{B \times n_E \times n_{aE}^{\max}}$:

$$x_E^{\text{eff}} = x_E - \sum_{j=0}^{n_{aE}^{\max}-1} (c_E \odot \text{sfa_E_mask})_j \cdot a_{E,j}.$$

(**sfa_E_mask** zeros out inactive tiers.) For the I partition ($n_E : N$), analogously with c_I , **sfa_I_mask**, a_I . Concatenate $x^{\text{eff}} = [x_E^{\text{eff}} | x_I^{\text{eff}}]$.

5.3 Firing rate

$$r = \sigma_{\text{pw}}(x^{\text{eff}} - a_0),$$

where $\sigma_{\text{pw}} : \mathbb{R} \rightarrow [0, 1]$ is the *piecewise sigmoid* (see §8).

5.4 Synaptic output with depression

Define the per-side gated availability

$$b_E^{\text{full}} = b_E \cdot \text{std_E_mask} + (1 - \text{std_E_mask}), \quad b_I^{\text{full}} \text{ analogously,}$$

so when STD is off the b -side stays exactly 1. Concatenate $b^{\text{full}} = [b_E^{\text{full}} | b_I^{\text{full}}] \in \mathbb{R}^{B \times N}$ and form

$$br = b^{\text{full}} \odot r, \quad Wbr = (br)(W^{\text{eff}})^{\top}.$$

5.5 Dendritic dynamics

$$\dot{x} = \frac{1}{\tau_d}(-x + u + Wbr) \quad (\text{per-neuron } \tau_d \text{ via } \tau_d(k, i)).$$

5.6 SFA dynamics (per tier j)

For j such that **sfa_E_mask** $[k, 0, j] = 1$:

$$\dot{a}_{E,j} = \frac{1}{\tau_{a,E,j}}(-a_{E,j} + c_{0,E,j} + r_E)$$

with $r_E = r[:, : n_E]$ broadcast over the tier dimension. Inactive tiers keep $a_{E,j}$ at its previous value (the masking is applied to the *update* in the batched solver; in the single cell, inactive tiers simply don't exist).

The I side is identical with $E \rightarrow I$.

5.7 STD dynamics

For E neurons (active iff **std_E_mask** $[k] = 1$):

$$\dot{b}_E = \frac{1 - b_E}{\tau_{b,\text{rec},E}} - \frac{r_E b_E}{\tau_{b,\text{rel},E}}$$

clamped to $[0, 1]$ after each step. The I side is identical with $E \rightarrow I$.

6. Solvers

The forward call performs n_{ode} sub-steps of size $\Delta t = h/n_{\text{ode}}$. Four discretizations are implemented; the default and most-tested is **semi_implicit**.

6.1 Semi-implicit (linearly-implicit Euler) — default

Let $\alpha_x = \Delta t / \tau_d$. Compute x^{eff} , r , b^{full} , Wbr as in §5.

$$x \leftarrow \frac{x + \alpha_x (u + Wbr)}{1 + \alpha_x}.$$

For SFA (active tiers only), $\alpha_a = \Delta t / \tau_a$:

$$a \leftarrow \frac{a + \alpha_a (c_0 + r)}{1 + \alpha_a}.$$

For STD (active sides only):

$$b \leftarrow \frac{b + \Delta t / \tau_{\text{rec}}}{1 + \Delta t (1 / \tau_{\text{rec}} + r / \tau_{\text{rel}})}, \quad b \leftarrow \text{clip}(b, 0, 1).$$

6.2 Explicit Euler (solver=explicit)

$$x \leftarrow x + \Delta t \dot{x}, \quad a \leftarrow a + \Delta t \dot{a}, \quad b \leftarrow \text{clip}(b + \Delta t \dot{b}, 0, 1).$$

6.3 Classical RK4 (solver=rk4)

Standard four-stage average using `_compute_rhs` to evaluate $(x, a_E, \dot{a}_I, b_E, b_I)$ at $0, \Delta t/2, \Delta t/2, \Delta t$. b states clamped to $[0, 1]$ post-update.

6.4 Exponential Euler (solver=exponential)

For the linear part: $\gamma_x = e^{-\Delta t / \tau_d}$, $x \leftarrow \gamma_x x + (1 - \gamma_x)(u + Wbr)$. SFA states use the same exponential update with $\gamma_a = e^{-\Delta t / \tau_a}$ toward target $c_0 + r$. STD uses explicit Euler (the equation is bilinear in b).

7. Activation function — piecewise sigmoid

Implemented in `piecewise_sigmoid(x; S_a, S_c)` with defaults $S_a = 0.9$, $S_c = 0$. Let $a = S_a/2$, $c = S_c$, and (when $|1 - 2a| > 10^{-8}$) $\kappa = 0.5/(1 - 2a)$. Define knot points

$$x_1 = c + a - 1, \quad x_2 = c - a, \quad x_3 = c + a, \quad x_4 = c + 1 - a.$$

Then

$$\sigma_{\text{pw}}(x) = \begin{cases} 0, & x < x_1 \\ \kappa(x - x_1)^2, & x_1 \leq x < x_2 \\ (x - c) + 1/2, & x_2 \leq x \leq x_3 \\ 1 - \kappa(x - x_4)^2, & x_3 < x \leq x_4 \\ 1, & x > x_4 \end{cases}$$

i.e. linear in the central region of width S_a , with quadratic shoulders joining 0 on the left and 1 on the right at x_1 and x_4 respectively.

readout	output
---------	--------

8. Cell-level readout (per variant)

Inside the cell, one of three views of the state is exposed as the per-timestep output (selected per variant via `readout_ids`):

readout	output
<code>synaptic</code> (id 0)	$b_{\text{last}}^{\text{full}} \odot r_{\text{last}}$
<code>rate</code> (id 1)	r_{last}
<code>dendritic</code> (id 2)	x_{last}

Output shape (K, B, N) in batched mode, (B, N) in single mode.

9. SequenceModel head

`SequenceModel` unrolls the cell over time and applies output masking, linear readout, and an optional skip residual.

9.1 Unrolling

Given input $X \in \mathbb{R}^{B \times T \times D}$ and an initial state s_0 (either `TrainableIC` or zeros), iterate $(o_t, s_t) = \text{cell}(X_{:,t,:}, s_{t-1})$ for $t = 0, \dots, T-1$ and stack outputs along the time axis to yield $\mathbf{O}$ of shape (B, T, N) (or (K, B, T, N) in batched mode). Truncated BPTT detaches state every `bptt_chunk_len` steps; the warmup region $t < \text{bptt_start_idx}$ runs under `torch.no_grad()`. `grad_checkpoint=True` wraps each segment in `torch.utils.checkpoint`.

9.2 Time-step selection

Pick a `readout_idx`:

- `int` (e.g. -1): output at one timestep, drop time axis.
- `slice`: keep a range of timesteps (used for multi-step regression losses).
- `None`: equivalent to -1.

Result $\mathbf{o} \in \mathbb{R}^{B \times N}$ (or $\mathbb{R}^{B \times T' \times N}$ for slices), batched-mode prepends K .

9.3 Output masking

`output_mask` is a binary $\{0, 1\}^N$ vector with $\sim N/4$ ones marking readout neurons. The output is filtered:

$$\mathbf{o}' = \mathbf{o}[\dots, \text{output_mask} = 1] \in \mathbb{R}^{\dots \times E},$$

where $E = \sum_i \text{output_mask}_i$ ($\approx N/4$).

9.4 Linear readout

Batched (K-mode):

$$\mathbf{y} = (W^{\text{ro}})\mathbf{o}'^T + b^{\text{ro}}, \quad W^{\text{ro}} \in \mathbb{R}^{K \times O \times E}, \quad b^{\text{ro}} \in \mathbb{R}^{K \times 1 \times O}.$$

Implemented as `einsum("k...e, koe -> k...o")` so it works for both (K, B, E) and (K, B, T', E). Non-batched mode uses one `nn.Linear(E, O)`.

9.5 Skip / residual (autoregressive variants)

When `cell.skip_flags[k] = 1` (else zero), and the input feature dim equals the output dim ($D = O$):

$$\mathbf{y} \leftarrow \mathbf{y} + \text{skip_flags}_k \cdot X_{:, \text{readout_idx}, :}$$

Effectively $y = \text{readout}(\text{state}) + x_{\text{at readout}}$ for skip variants, asking the network to learn the *residual* signal — natural for SEEG-style autoregressive forecasting where the next sample is mostly the current sample.

10. Forward pipeline (one minibatch)

Putting it together for a batched forward call:

1. **Initial state.** $s_0 \leftarrow \text{TrainableIC}()$ or zeros.
2. **Warmup (no grad).** For $t = 0$ to `bptt_start_idx - 1`, advance the cell.
3. **Grad region.** For $t = \text{bptt_start_idx}$ to $T - 1$:
 - **Drive.** $u_t = W^{\text{in, eff}} X_{:, t, :}$.
 - **ODE.** Run n_{ode} sub-steps of the chosen solver, each implementing equations §5–6.
 - **Per-cell readout.** Apply per-variant readout selection (synaptic/rate/dendritic) $\rightarrow o_t$.
4. **Time-step pick.** $\mathbf{o} = \mathbf{O}[\dots, \text{readout_idx}, :]$.
5. **Output mask.** Project to readout neurons $\rightarrow \mathbf{o}' \in \mathbb{R}^{\dots \times E}$.
6. **Linear head.** $\mathbf{y} = W^{\text{ro}} \mathbf{o}' + b^{\text{ro}}$.
7. **Skip residual.** If `skip_flags[k] = 1`: $\mathbf{y} \leftarrow \mathbf{y} + X_{:, \text{readout_idx}, :}$.

Total trainable parameter count, batched, with `per_neuron=False` on a typical variant slice: $\Theta_k \subset \{W^{\text{raw}}, W^{\text{in}}, W_{\text{gain}}^{\text{raw}}, W_{\text{gain}}^{\text{in}}, \log \tau_g, \log \tau_d\}$

11. Ablation knobs (which parts of §2–§9 are gated)

Each `SRNNConfig` flag affects the model as follows. In `BatchedSRNNCell`, all variants live in the same parameter tensors; the gating is via masks and gradient hooks rather than removing parameters.

11.1 dales (default True)

Selects branch in §4.1: with Dale's, W^{eff} has column-wise sign-pinned softplus; without Dale's, W^{eff} uses raw W^{raw} directly. `dales_signs`, `sparsity_masks` are still applied either way.

11.2 echo (default False) — reservoir mode

Freezes W^{raw} via the gradient hook `W_raw.register_hook(grad  $\rightarrow$  grad  $\cdot$  _echo_grad_mask)` where `_echo_grad_mask = 1 - echo_flags`. **Note:** `W_raw_gain` is *not* explicitly frozen, so a scalar rescale of the reservoir is still learnable (it modulates the spectral radius). All other parameters (`taus`, `thresholds`, W^{in} , `readout`) train normally.

11.3 per_neuron (default False)

For non-per-neuron variants, gradient hooks installed on every per-neuron `*_vec` parameter multiply the gradient by `per_neuron_flags[k]`, freezing the vec at its identical-across-neurons init. The `*_gain` and `*_scalar` companions remain trainable, supplying a single shared per-variant direction. For per-neuron variants, both halves train; the scalar/gain captures the shared direction and the vec captures per-neuron deviations.

11.4 n_a_E, n_a_I $\in \{0, 1, \geq 2\}$

- $n_a = 0$: SFA off — `sfa*_mask = 0`, the SFA contribution to x^{eff} is zero, and the SFA update is masked out (state stays at init, but is also irrelevant).
- $n_a = 1$: single-tier SFA at $\tau \approx 1$ s init.
- $n_a \geq 2$: multi-timescale SFA. In the batched cell, n_a tiers are stored along the trailing dim and `sfa*_mask` selects the active tiers. In the single cell, an interpolation between learnable lo/hi endpoints generates n_a tiers at runtime.

11.5 `n_b_E, n_b_I` $\in \{0, 1\}$

- $n_b = 0$: STD off — `std_*_mask` = 0, b^{full} stays at 1, the $\dot{b}$ update is gated out.
- $n_b = 1$: STD on, with recovery time τ_{rec} and release/depression time τ_{rel} .

11.6 `skip` (default `False`)

Activates the post-readout residual described in §9.5.

11.7 `solver` $\in \{\text{semi_implicit}, \text{explicit}, \text{rk4}, \text{exponential}\}$

Selects discretization; semantics and stability differ but the underlying ODE in §5 is the same.

11.8 `readout` $\in \{\text{synaptic}, \text{rate}, \text{dendritic}\}$

Selects the per-cell output (§8).

11.9 `tau_global_init` (default 1.0)

Sets the init of `log_tau_global` to $\text{softplus}^{-1}(\tau_g^{\text{init}})$.

12. Loss and training (out-of-cell, for completeness)

The training script `train.py` computes the loss between model logits at `readout_idx` (a single timestep, or a slice for per-timestep tasks) and the corresponding label slice from `batch_y`. Adam with `WarmupHoldCosineSchedule` (per-step: 20 % warmup capped at 3 epochs, 70 % hold, 10 % cosine). After each optimizer step, `model.constrain_parameters()` applies any cell-specific constraints (no-op for SRNN; LTC clips weights). Mixed precision (`amp=bf16` on CUDA) wraps only the forward + loss; optimizer state stays fp32.

13. Symbol $\rightarrow$ parameter map (cheat sheet)

math	code (<code>cell.*</code> unless noted)	shape	trainable?
W^{raw}	<code>W_raw</code>	(K, N, N)	yes (frozen iff echo)
g_W	<code>W_raw_gain</code>	$(K,)$	yes
W^{in}	<code>W_in</code>	(K, N, D)	yes
$g_{W_{\text{in}}}$	<code>W_in_gain</code>	$(K,)$	yes
a_0	<code>a_0_vec + a_0_scalar</code>	$(K, N) + (K,)$	vec frozen iff not <code>per_neuron</code>
τ_g	<code>softplus(log_tau_global)</code>	$(K,)$	yes
τ_d	<code>$\tau_g \cdot \exp(\log_tau_d_gain) \cdot \text{softplus}(\log_tau_d_vec)$</code>	(K, N)	gain yes; vec only if <code>per_neuron</code>
$\tau_{a,E}^{(j)}$	<code>$\tau_g \cdot \exp(\log_tau_a_E_gain) \cdot \text{softplus}(\log_tau_a_E_vec_{[:,j]})$</code>	$(K, n_E, n_{aE}^{\text{max}})$	as above; tier active iff <code>sfa_E_mask[k, 0, j]=1</code>
c_E	<code>$\exp(\log_c_E_gain) \cdot \text{softplus}(\log_c_E_vec)$</code>	$(K, n_E, n_{aE}^{\text{max}})$	as above
$c_{0,E}$	<code>c_0_E_vec + c_0_E_scalar</code>	same	vec frozen iff not <code>per_neuron</code>
$\tau_{b,\text{rec},E}$	<code>$\tau_g \cdot \exp(\log_tau_b_rec_E_gain) \cdot \text{softplus}(\log_tau_b_rec_E_vec)$</code>	(K, n_E)	as above
$\tau_{b,\text{rel},E}$	analogous	(K, n_E)	as above
$W^{\text{ro}}, b^{\text{ro}}$	<code>readout_weight,</code> <code>readout_bias</code> (in <code>SequenceModel</code>)	$(K, O, E), (K, 1, O)$	yes
ic	<code>ic.ic</code>	(K, S)	yes (optionally frozen post burn-in)

(Replace `_E` with `_I` for the inhibitory side; structure identical.)